



Universidad Autónoma del Estado de México

Centro Universitario UAEM Zumpango
Ingeniería en Computación

Reporte técnico

Materia: Sistemas Operativos

Alumno: García Pérez Leonel

Fecha: 2026

1. Introducción

Este reporte documenta el proceso que seguí para cumplir con la tarea asignada en clase: crear un repositorio en GitHub, sincronizarlo y desarrollar un pequeño programa en Python capaz de detectar los bordes de una imagen usando la librería OpenCV. Más que una lista de comandos, la idea es mostrar el razonamiento detrás de cada paso: por qué se hace, qué resuelve y cómo se comprobó que funcionó correctamente antes de pasar al siguiente.

2. Herramientas utilizadas

Para el desarrollo de esta actividad utilicé: **GitHub** como plataforma de alojamiento del repositorio remoto, la **consola de comandos (CMD)** y **Git Bash** para gestionar el repositorio local, y **Python 3** junto con la librería **OpenCV** para el procesamiento de la imagen.

3. Desarrollo

3.1 Creación del repositorio en GitHub

El primer paso fue crear el repositorio. Desde GitHub abrí la opción de nuevo repositorio y lo nombré siguiendo lo solicitado, DevOps_ApellidoPaterno Nombre, quedando como “DevOps_GarciaLeonel”. Lo configuré como público, sin archivo README inicial ni licencia, para partir de un repositorio completamente vacío y agregar el contenido yo mismo desde la terminal más adelante.

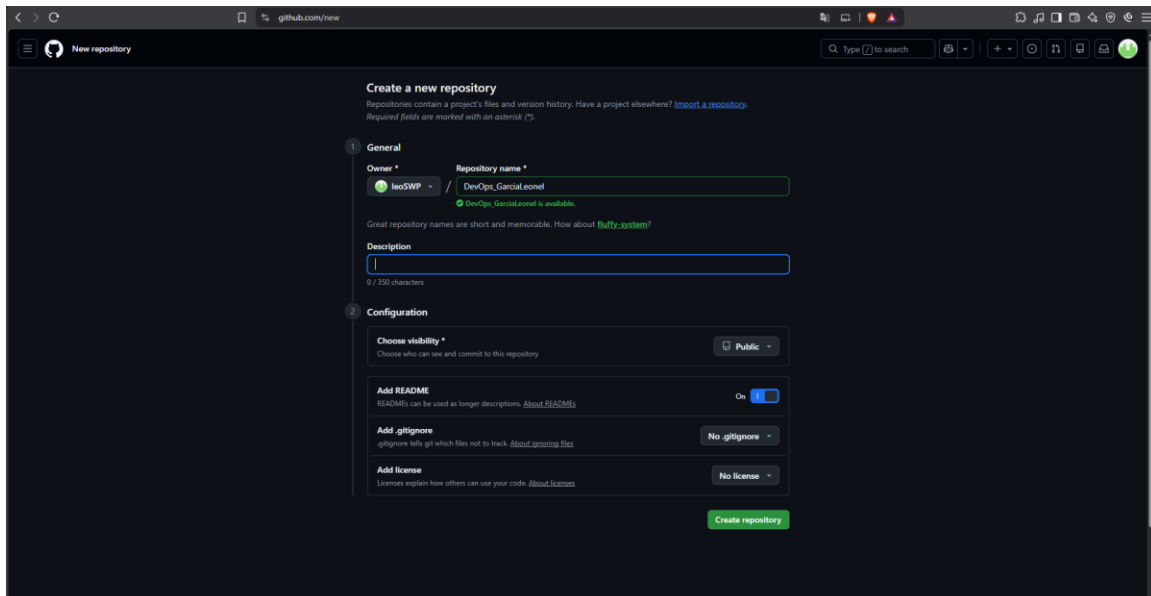


Figura 1. Creación del repositorio DevOps_GarciaLeonel en GitHub.

3.2 Creación de la carpeta de trabajo mediante CMD

Con el repositorio listo, el siguiente paso fue preparar el espacio local. Abrí la consola de comandos, me ubiqué en la carpeta Documentos y utilicé mkdir para crear la carpeta del proyecto, nombrada MiProyectoSO seguida de las iniciales de mi apellido paterno y nombre. Después usé dir para confirmar que la carpeta se había creado correctamente entre el resto de los directorios existentes.

```

C:\Users\Family fresh\Documents>mkdir MiProyectoSO_PL

C:\Users\Family fresh\Documents>dir
El volumen de la unidad C no tiene etiqueta.
El número de serie del volumen es: A048-30C5

Directorio de C:\Users\Family fresh\Documents

14/08/2026  11:10    <DIR>          .
14/08/2026  11:10    <DIR>          ..
17/01/2025  09:30             470,206 (VACIO)Formato de registro de UA 2025A.docx
13/08/2026  22:30    <DIR>          11Leonel
17/10/2024  22:21    <DIR>          Adobe
06/11/2020  19:32    <DIR>          Adobe Dreamweaver 2021
28/06/2025  23:33    <DIR>          bnvbnbnv
17/10/2025  10:09             14,753 Codigo_EnsambladorPrac2.docx
11/08/2024  00:25    <DIR>          EvidenciaCaso
17/01/2025  09:31             470,504 Formato de registro de UA 2025A.docx
19/10/2024  20:37    <DIR>          Fritzing
23/07/2023  18:09    <DIR>          IDB8.2
01/02/2021  17:31    <DIR>          Jueguitos jeje
08/05/2025  15:54    <DIR>          KiCad
17/10/2024  22:23    <DIR>          Lightshot
30/01/2023  20:15    <DIR>          maya
14/08/2026  11:10    <DIR>          MiProyectoSO_PL

```

Figura 2. Creación de la carpeta MiProyectoSO_PL mediante el comando mkdir.

4.3 Inicialización del repositorio local

Una vez dentro de la carpeta, la inicialicé como repositorio de Git con git init. Verifiqué el estado con git status, confirmando que el repositorio quedó en la rama master, sin commits todavía y sin archivos pendientes, es decir, listo para empezar a trabajar.

```

MINGW64/c/Users/Family fresh/Documents/MiProyectoSO_PL

Family fresh@DESKTOP-3COQUQ6 MINGW64 ~/Documents/MiProyectoSO_PL
$ git init
Initialized empty Git repository in C:/Users/Family fresh/Documents/MiProyectoSO_PL/.git/

Family fresh@DESKTOP-3COQUQ6 MINGW64 ~/Documents/MiProyectoSO_PL (master)
$ git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)

Family fresh@DESKTOP-3COQUQ6 MINGW64 ~/Documents/MiProyectoSO_PL (master)
$ |

```

Figura 3. Inicialización del repositorio local con git init y verificación de su estado.

4.4 Sincronización del repositorio local con el remoto

Para conectar el repositorio local con el que había creado en GitHub, primero renombré la rama principal de master a main con git branch -M main. Después vinculé ambos repositorios con git remote add origin, apuntando a la URL del repositorio DevOps_GarciaLeonel. Finalmente comprobé que la conexión quedara bien establecida ejecutando git remote -v, que mostró correctamente las direcciones de fetch y push apuntando al mismo repositorio.

```

Family fresh@DESKTOP-3COQUQ6 MINGW64 ~/Documents/MiProyectoSO_PL (master)
$ git branch -M main

Family fresh@DESKTOP-3COQUQ6 MINGW64 ~/Documents/MiProyectoSO_PL (main)
$ git remote add origin https://github.com/leoSWP/DevOps_GarciaLeonel.git

Family fresh@DESKTOP-3COQUQ6 MINGW64 ~/Documents/MiProyectoSO_PL (main)
$ |

```

Figura 4. Cambio de rama a main y vinculación con el repositorio remoto (git remote add origin).

```
Family fresh@DESKTOP-3COQUQ6 MINGW64 ~/Documents/MiProyectoSO_PL (main)
$ git remote -v
origin https://github.com/leoSWP/DevOps_GarciaLeonel.git (fetch)
origin https://github.com/leoSWP/DevOps_GarciaLeonel.git (push)

Family fresh@DESKTOP-3COQUQ6 MINGW64 ~/Documents/MiProyectoSO_PL (main)
$
```

Figura 5. Verificación de la conexión remota con git remote -v.

4.5 Desarrollo del programa en Python para detección de bordes

Con el repositorio ya sincronizado, regresé a la consola para crear el programa solicitado. Usando notepad bordes.py desde CMD generé el archivo, y ahí use un script en Python apoyado en la librería OpenCV. El programa carga una imagen (en este caso una fotografía llamada Daft-Punk-1.jpg), valida que se haya leído correctamente, la convierte a escala de grises y aplica el algoritmo de Canny para detectar sus bordes. El resultado se guarda como archivo de imagen y también se muestra en pantalla junto con la imagen original, para poder comparar visualmente el efecto del procesamiento.



Figura 6. Creación del archivo bordes.py desde la consola.

```
bordes.py 1 X
C: > Users > Family fresh > Documents > MiProyectoSO_PL > bordes.py > ...
1 import cv2
2
3 # Nombre de la imagen original
4 nombre_imagen = "Daft-Punk-1.jpg"
5
6 # Cargar la imagen
7 imagen = cv2.imread(nombre_imagen)
8
9 # Verificar que la imagen se haya cargado correctamente
10 if imagen is None:
11     print("Error: no se pudo cargar la imagen.")
12     exit()
13
14 # Convertir la imagen a escala de grises
15 gris = cv2.cvtColor(imagen, cv2.COLOR_BGR2GRAY)
16
17 # Detectar los bordes utilizando Canny
18 bordes = cv2.Canny(gris, 100, 200)
19
20 # Guardar el resultado
21 cv2.imwrite("resultado_bordes.png", bordes)
22
23 # Mostrar las imágenes
24 cv2.imshow("Imagen original", imagen)
25 cv2.imshow("Bordes", bordes)
26
27 print("Los bordes fueron detectados correctamente.")
28 print("Resultado guardado como: resultado_bordes.png")
29
30 # Esperar una tecla
31 cv2.waitKey(0)
32
33 # Cerrar las ventanas
34 cv2.destroyAllWindows()
```

Figura 7. Código completo del programa bordes.py: lectura de la imagen, conversión a escala de grises y detección de bordes con el algoritmo de Canny.

4.6 Ejecución del programa y almacenamiento del resultado

Ejecuté el script con python bordes.py y la consola confirmó que los bordes se detectaron correctamente, guardando el resultado como imagen. Al abrir la ventana generada por el propio

programa pude ver el resultado del algoritmo de Canny: una imagen en blanco y negro donde las líneas blancas resaltan los contornos y cambios bruscos de intensidad de la fotografía original, mientras que las zonas más uniformes quedan en negro. Después, siguiendo el formato pedido en la tarea, renombré el archivo de salida a result_GL.png (iniciales de mi apellido paterno y nombre), lo cual puede confirmarse en el explorador de archivos junto al resto de los archivos del proyecto: el script bordes.py, la imagen original Daft-Punk-1 y el resultado ya renombrado.

```
C:\Users\Family_fresh\Documents\MiProyectoSO_PL>python bordes.py
Los bordes fueron detectados correctamente.
Resultado guardado como: resultado_bordes.png
```

Figura 8. Ejecución del programa desde CMD y confirmación en consola.

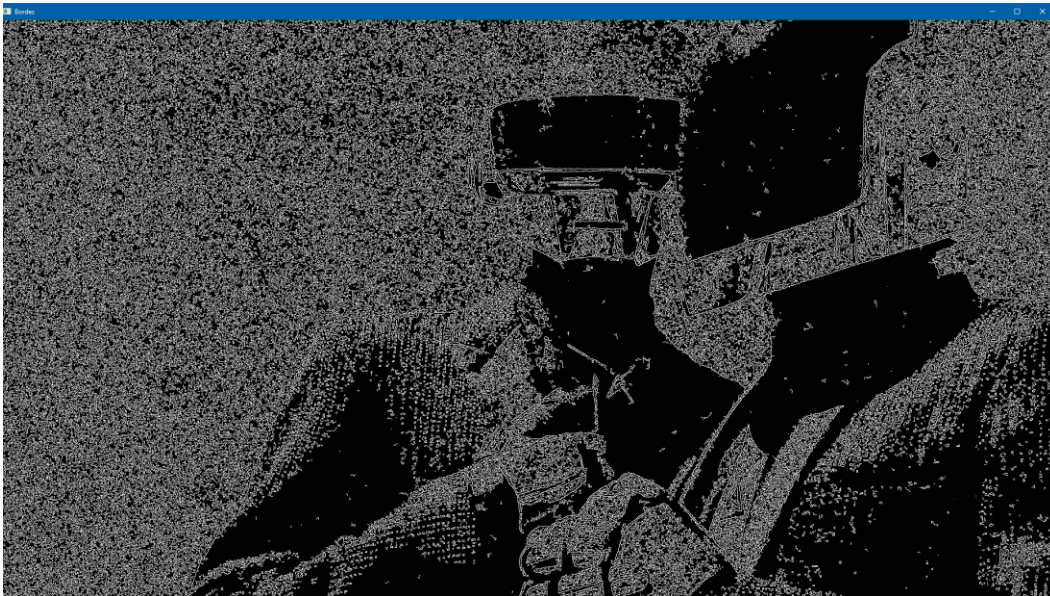


Figura 9. Resultado de la detección de bordes mostrado en la ventana generada por OpenCV.

Nombre	Fecha de modificación	Tipo	Tamaño
.git	14/8/2026 11:14	Carpeta de archivos	
bordes	14/8/2026 11:41	Archivo de origen ...	1 KB
Daft-Punk-1	14/8/2026 11:33	Archivo JPG	964 KB
result_GL	14/8/2026 11:41	Archivo PNG	303 KB

Figura 10. Archivos del proyecto en el explorador, incluyendo el resultado renombrado como result_GL.png.

4.7 Subida de los archivos al repositorio en GitHub

Como paso final, subí todo el trabajo al repositorio remoto. Añadí los archivos al área de preparación con `git add .`, los confirmé con `git commit -m "detección de bordes"`, y utilicé `git push` para enviarlos a GitHub; la subida se completó sin problemas: los tres archivos del proyecto (Daft-Punk-1.jpg, bordes.py y result_GL.png) quedaron reflejados en el repositorio DevOps_GarciaLeonel de GitHub.

```

Family fresh@DESKTOP-3COQUQ6 MINGW64 ~/Documents/MiProyectoSO_PL (main)
$ git add .

Family fresh@DESKTOP-3COQUQ6 MINGW64 ~/Documents/MiProyectoSO_PL (main)
$ git commit -m "Programa para detectar bordes"
[main (root-commit) dc12d1b] Programa para detectar bordes
 3 files changed, 34 insertions(+)
 create mode 100644 Daft-Punk-1.jpg
 create mode 100644 bordes.py
 create mode 100644 result_GL.png

Family fresh@DESKTOP-3COQUQ6 MINGW64 ~/Documents/MiProyectoSO_PL (main)
$ git push
fatal: The current branch main has no upstream branch.
To push the current branch and set the remote as upstream, use

    git push --set-upstream origin main

To have this happen automatically for branches without a tracking
upstream, see 'push.autoSetupRemote' in 'git help config'.

```

Figura 11. Primer intento de subida; Git señala que falta configurar la rama upstream.

```

Family fresh@DESKTOP-3COQUQ6 MINGW64 ~/Documents/MiProyectoSO_GL (main)
$ git add .

Family fresh@DESKTOP-3COQUQ6 MINGW64 ~/Documents/MiProyectoSO_GL (main)
$ git commit -m "deteccion de bordes"
[main c81a1a0] deteccion de bordes
 3 files changed, 34 insertions(+)
 create mode 100644 Daft-Punk-1.jpg
 create mode 100644 bordes.py
 create mode 100644 result_GL.png

Family fresh@DESKTOP-3COQUQ6 MINGW64 ~/Documents/MiProyectoSO_GL (main)
$ git push
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 1.18 MiB | 1.01 MiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/leoSWP/DevOps_GarciaLeonel.git
 225bed2..c81a1a0  main -> main

```

Figura 12. Subida exitosa de los archivos al repositorio remoto tras configurar el upstream.

5. Resultados

Al finalizar la actividad, el repositorio DevOps_GarciaLeonel en GitHub contiene la carpeta de trabajo completa: el script bordes.py, la imagen original utilizada como entrada y la imagen resultante result_GL.png con la detección de bordes aplicada. El historial de commits refleja de manera ordenada cada etapa del proceso, desde la inicialización del repositorio hasta la subida final del resultado, lo que permite dar seguimiento a todo el trabajo realizado.

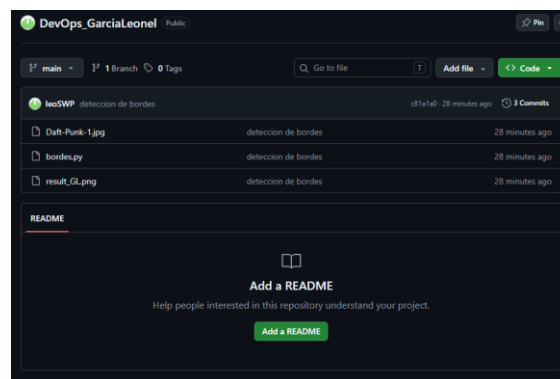


Figura 13. Los archivos reflejados en GitHub

6. Conclusión

Esta actividad me permitió integrar dos habilidades que normalmente se practican por separado: el manejo de un flujo de trabajo con Git y GitHub, y la programación aplicada a procesamiento de imágenes. Más allá de memorizar comandos, quedó claro el orden lógico que sigue todo este proceso: primero se define dónde va a vivir el proyecto, luego se prepara el entorno local y se conecta con ese repositorio, y solo hasta entonces tiene sentido empezar a generar contenido, en este caso un programa funcional en Python.